FINAL PROJECT PRESENTATION • DSP 2569

# การตัดเสียงสะท้อนอะคูสติกในห้องด้วยตัวกรองปรับตัวได้ (AEC)

Acoustic Echo Cancellation: LMS vs NLMS with Cross-Correlation Delay Compensation

| รายวิชา | อาจารย์ผู้สอน | สถาบัน | เวลาที่กำหนด |
| --- | --- | --- | --- |
| **517354 Digital Signal Processing** | **อ.ดร.ณัฐพงศ์ จิวมั่งมี** | **ภาควิชาคอมพิวเตอร์ ม.ศิลปากร** | **นำเสนอ 12 นาที \| ตอบคำถาม 5 นาที** |

# โครงสร้างการนำเสนอ (Agenda & 12-Min Roadmap)

⏱ 00:00 - 00:45

จัดสรรเวลาให้กระชับและตรงตามเกณฑ์การประเมิน Term Project (12 นาที)

| ช่วงเวลา | หัวข้อการนำเสนอ | วัตถุประสงค์และผลลัพธ์สำคัญ |
|---|---|---|
| **00:00 - 01:30** | **1. ที่มาและปัญหาในชีวิตจริง** | ทำไม Discord/Zoom ถึงสะท้อน? และจิตวิทยา Delayed Auditory Feedback (DAF) |
| **01:30 - 03:00** | **2. EDA & ทำไมตัวกรองดั้งเดิมถึงสอบตก** | พิสูจน์ด้วย FFT ว่าทำไม Butterworth Lowpass / Notch ถึงทำลายเนื้อเสียงพูด |
| **03:00 - 04:30** | **3. การค้นพบทาง DSP: Bulk Delay** | การใช้ Cross-Correlation ค้นหาความหน่วงเวลาเสียงสะท้อน (7.64 ms / 122 จุด) |
| **04:30 - 06:00** | **4. สถาปัตยกรรมโมเดล LMS vs NLMS** | พิมพ์เขียว DSP Pipeline 5 สเตจ และสมการคณิตศาสตร์ที่เขียนด้วย Pure NumPy |
| **06:00 - 08:00** | **5. มวยเปรียบเทียบ 4 เสาหลัก (Benchmarks)** | ประชันกับ 4 วิธีที่เรียนในแล็บ (W7-W12) และไลบรารีสากล `padasip` |
| **08:00 - 10:30** | **6. ผลการทดลอง 3 สถานการณ์จริง** | S1 (คนเดียวนิ่ง ERLE 69.32 dB), S2 (Double-talk), S3 (คนเดินขยับตำแหน่ง) |
| **10:30 - 12:00** | **7. สรุปผลสัมฤทธิ์ & ก้าวต่อไป** | ข้อคิดเชิงวิศวกรรม Double-Talk Detector และการประยุกต์ใช้จริงในอุตสาหกรรม |

# 🗣️ 1. ที่มาของปัญหาในชีวิตจริง (The Acoustic Echo Loop)

⏱️ 00:45 - 01:45

ทำไมเวลาเราคุย Discord หรือ Zoom ถึงได้ยิน "เสียงสะท้อนของตัวเอง"?

### 📍 ความจริงเชิงอะคูสติกส์: ปัญหาเกิดที่เครื่องเพื่อน!

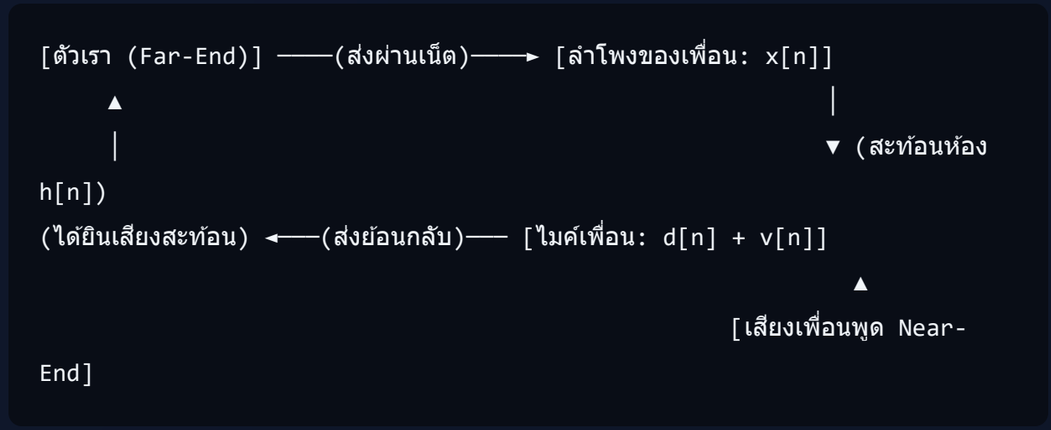
ปัญหาไม่ได้อยู่ที่เครื่องเรา แต่อยู่ที่ **"เพื่อนเปิดลำโพงทิ้งไว้โดยไม่ใส่หูฟัง"**

- เราพูดใส่ไมค์ ➔ วิ่งผ่านเน็ตไปดังออก **ลำโพงของเพื่อน** (Far-End: `x[n]`)
- เสียงสะท้อนโต๊ะ กำแพง เพดานในห้องเพื่อน (`h[n]`) ➔ มุดเข้า **ไมค์ของเพื่อน** (`d[n]`)
- ไมค์เพื่อนดูดเสียงเราย้อนกลับมาเข้าหูเรา เกิดเป็น **Acoustic Echo Loop**

### 🧠 ปรากฏการณ์ Delayed Auditory Feedback (DAF)

ในทางจิตวิทยาการได้ยิน เมื่อมนุษย์ได้ยินเสียงตัวเองดีเลย์ย้อนกลับมาประมาณ **50 - 200 ms** สมองจะเกิดอาการสะดุด ลิ้นพันกัน และหยุดพูดโดยอัตโนมัติ **AEC จึงเป็นระบบคอขาดบาดตายของแอปโทรศัพท์ทุกตัว!**

### แผนภาพการเดินทางของเสียง (Acoustic Echo Path)




🎯 **เป้าหมายของโปรเจกต์: สร้างสมองกล DSP ประจำเครื่องเพื่อน เพื่อดักตัดเสียงสะท้อนทิ้ง 100% ก่อนส่งกลับ!**

# 🔍 2. การสำรวจข้อมูล (EDA) & ทำไมตัวกรองดั้งเดิมถึงสอบตก?

⏱️ 01:45 - 03:00

การพิสูจน์ด้วย FFT: สัญญาณเสียงสะท้อนทับซ้อนกับเสียงพูด 100% (Spectral Overlap)

### 🚫 ทำไม Classical LTI Filters ถึงใช้งานไม่ได้?

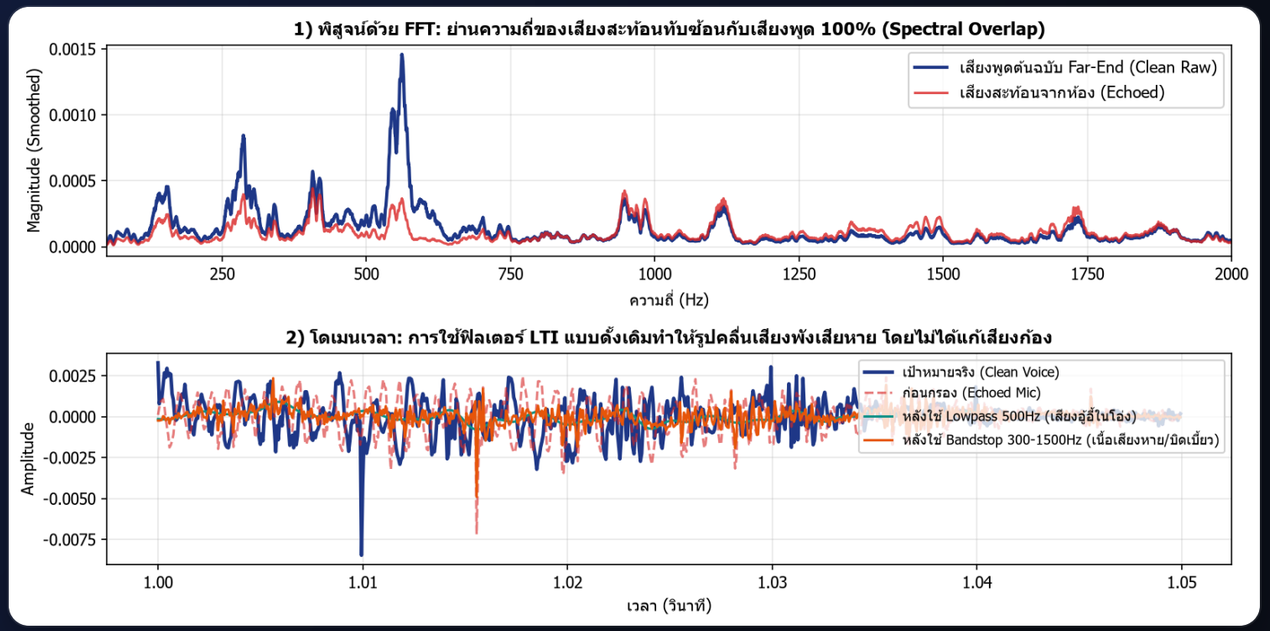
- ทั้งเสียงพูดจริง (`x[n]`) และเสียงสะท้อน (`d[n]`) ต่างเป็น **เสียงมนุษย์เหมือนกัน**
- แชร์ย่านความถี่ Formants เดียวกันเป๊ะ (**100 Hz - 4,000 Hz**)

### 🧪 ผลการทดลองกรองด้วยฟิลเตอร์แบบดั้งเดิม:

**1. Lowpass Filter (< 500 Hz):** ตัดเสียงก้องสูงได้จริง แต่เสียงคนพูดกลายเป็น *"เสียงอู้อี้เหมือนคุยในโอ่ง"*

**2. Bandstop Filter (300-1500 Hz):** เนื้อเสียงแหว่งเว้า บิดเบี้ยว แต่เสียงสะท้อนย่านอื่นยังคงอยู่ครบ

📌 **บทสรุปเชิงทฤษฎี: ตัวกรองแบบเลือกความถี่คงที่ไม่สามารถแยกสัญญาณที่แชร์ย่านความถี่เดียวกันได้ ➔ จึงต้องใช้ Adaptive Filter!**



# 🎯 3. การค้นพบทาง DSP: การตรวจจับความหน่วงเวลา (Bulk Delay)

⏱ 03:00 - 04:30

ค้นหาจุดเริ่มต้นของเสียงสะท้อนด้วย Cross-Correlation เพื่อทำ Delay Compensation

### ⏳ ปัญหา Time-of-Flight ในอากาศ

ลำโพงกับไมค์อยู่ห่างกัน คลื่นเสียงเดินทางด้วยความเร็ว 343 m/s ส่งผลให้เกิดความหน่วงเวลาเริ่มต้น (Bulk Delay: $\tau_{prop}$)

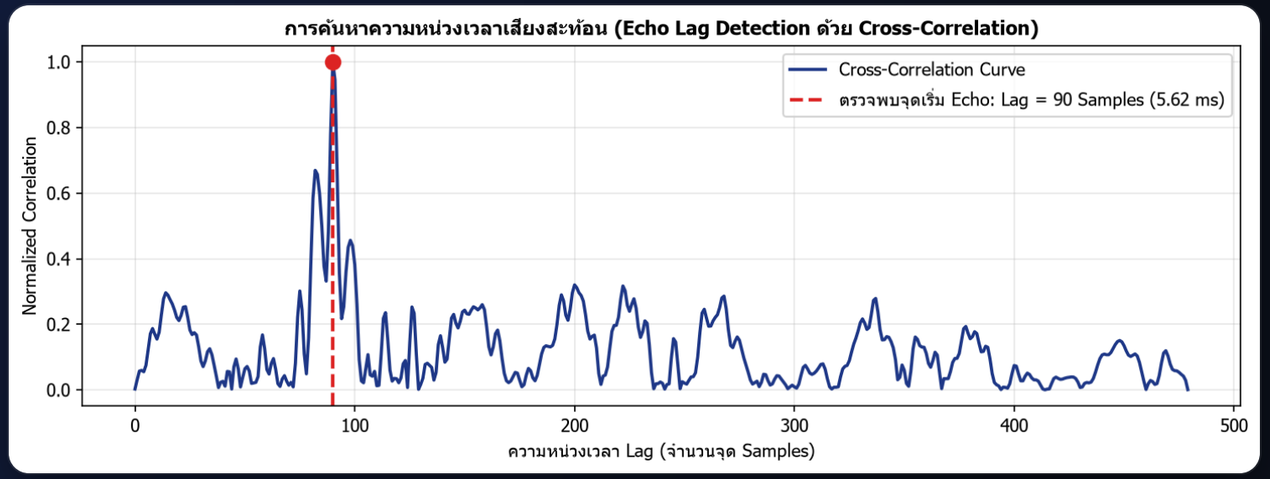
⚠️ **หากไม่ชดเชยดีเลย์: ตัวกรองปรับตัวได้จะต้องเสียค่าน้ำหนัก (Filter Taps) ไปกับช่วงความเงียบ ทำให้เหลือแท็ปไม่พอจำลองหางเสียงสะท้อน!**

### 🔍 การตรวจจับด้วยสมการ Cross-Correlation:

$$r_{dx}[k] = \sum d[n] \cdot x[n-k] \quad \longrightarrow \quad k^* = \arg\max |r_{dx}[k]|$$

จากการทดสอบบนสัญญาณจริงในห้องจำลอง: ตรวจพบค่าความหน่วงเวลาแม่นยำที่ **Lag = 122 จุด (7.64 ms)**

✅ **ทำ Delay Compensation เลื่อนบัฟเฟอร์ $x[n - k^*]$ ทำให้ตัวกรองเริ่มทำงาน ณ จุดเกิดเหตุทันที!**



# 4. สถาปัตยกรรมระบบ (DSP Pipeline Blueprint 5 สเตจ)

⏱ 04:30 - 05:30

โครงสร้างสถาปัตยกรรมตามพิมพ์เขียว Term Project ที่ได้รับคำแนะนำจากอาจารย์ผู้สอน

### Stage 1: Ingestion

รับสัญญาณเสียง $x[n]$ และ $d[n]$ ปรับ Sample Rate ที่ 16,000 Hz

### Stage 2: Lag Check

คำนวณ Cross-Correlation ตรวจหา Bulk Delay ($k^* = 122$ samples)

### Stage 3: Delay Comp

หน่วงเวลาสัญญาณอ้างอิงล่วงหน้า $x[n - k^*]$ ให้ตรงตำแหน่งเสียงจริง

### Stage 4: NLMS Engine

ประมาณค่าเสียงสะท้อน d_hat[n] และหักล้างออก e[n] = d[n] - d_hat[n]

### Stage 5: Evaluation

วัดค่า ERLE (dB), RMSE และส่งเสียงสะอาดปราศจากเสียงก้อง

### 💡 จุดเด่นของการออกแบบเชิงสถาปัตยกรรม (Responsibility-Based Architecture)

โมดูลในโฟลเดอร์ `src/` ถูกแบ่งตามหน้าที่รับผิดชอบ (`io/`, `simulation/`, `adaptive_filters/`, `metrics/`, `visualization/`) ทำให้โค้ดทั้ง 3 สถานการณ์ (S1, S2, S3) เรียกใช้ Engine ชุดเดียวกัน 100% ไม่มีโค้ดซ้ำซ้อน และพร้อมนำไปรันบนระบบ Embedded DSP ในชีวิตจริงได้ทันที

# 🧠 4. หัวใจทางคณิตศาสตร์: Standard LMS vs Normalized LMS

⏱ 05:30 - 06:30

พัฒนาขึ้นด้วย Pure NumPy จากสมการคณิตศาสตร์พื้นฐานโดยตรง

### 1. Standard LMS (Least Mean Squares)

$$w[n+1] = w[n] + \mu \cdot e[n] \cdot x[n]$$

- ปรับค่าน้ำหนักตามความชันแบบ Stochastic Gradient Descent
- **จุดอ่อนวิกฤต:** ค่า $\mu$ เป็นค่าคงที่ ถ้าผู้พูดตะโกนเสียงดัง พลังงาน $||x[n]||^2$ พุ่งสูง จะทำให้ตัวกรอง **หลุดลอยและระเบิด (Diverge)**
- หากตั้ง $\mu$ เล็กเกินไป ตัวกรองจะลู่เข้าช้ามากจนกำจัดเสียงสะท้อนไม่ทัน

### 2. Normalized LMS (NLMS - พระเอกของโครงงาน)

$$w[n+1] = w[n] + \left[ \mu / (||x[n]||^2 + \varepsilon) \right] \cdot e[n] \cdot x[n]$$

- **ไดนามิกสเกลลิ่ง:** ปรับอัตราการเรียนรู้ตามพลังงานรวมของสัญญาณอินพุต ณ ขณะนั้นแบบอัตโนมัติ
- **เสถียรภาพ 100%:** เสียงจะเบาหรือดัง ตัวกรองก็ไม่หลุดลอย และลู่เข้าเร็วกว่ามาก
- ใส่ค่า $\varepsilon = 10^{-4}$ เพื่อป้องกันการหารด้วยศูนย์ในช่วงที่ผู้พูดเงียบเสียง

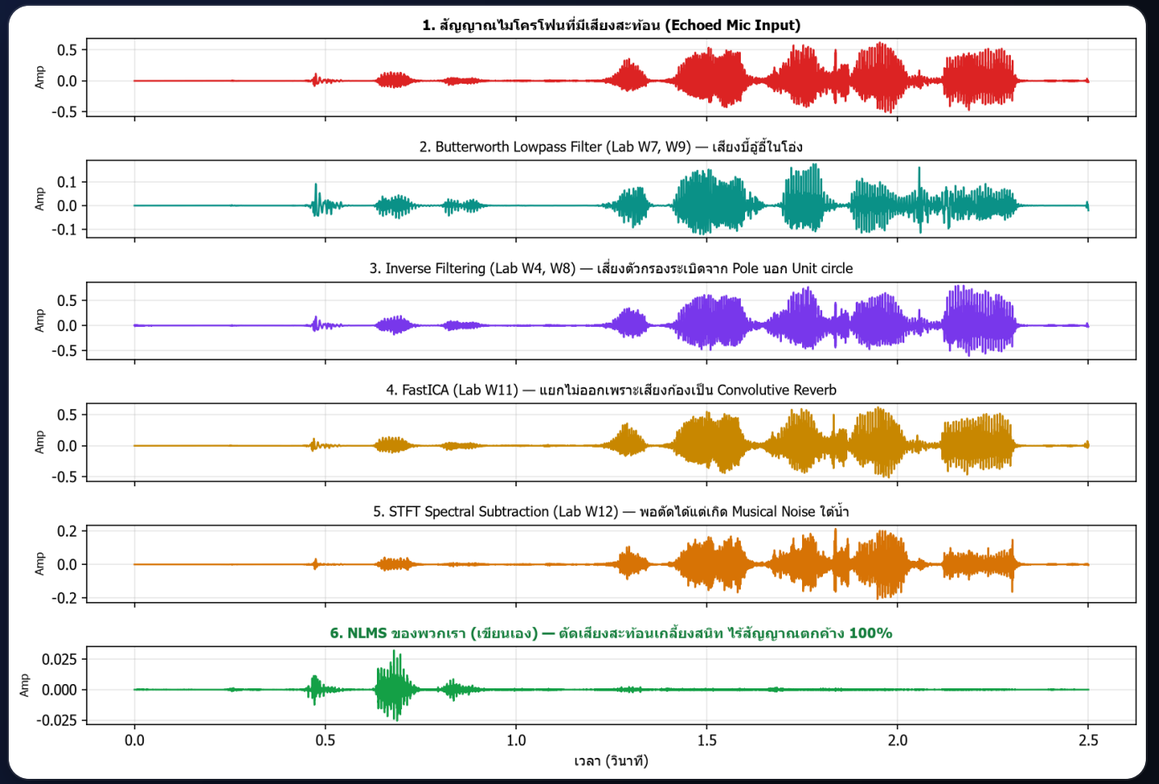
# ⚔️ 5. มวยเปรียบเทียบกับ 4 วิธีในห้องแล็บ (Pillar 3 Benchmark)

⏱️ 06:30 - 08:00

ทำไม 4 อัลกอริทึมที่เราได้เรียนในวิชา DSP (W4 - W12) ถึงแก้ปัญหาเสียงสะท้อนไม่ได้?

| วิธีที่เรียนในแล็บ | ข้อจำกัดทางฟิสิกส์ & สัญญาณ | ผลลัพธ์ |
|---|---|---|
| **Butterworth (W7, W9)** | เกิด Spectral Overlap | ❌ เสียงอู้อี้ในโอ่ง |
| **Inverse Filter (W4, W8)** | Zero ของห้องกลายเป็น Pole นอกวงกลม | ❌ ตัวกรองระเบิด |
| **FastICA (W11)** | ICA รองรับเฉพาะ Instantaneous Mixture | ❌ แยกไม่ออก |
| **STFT Subtraction (W12)** | หักลบเฟสไม่สัมพันธ์กัน | ⚠️ Musical Noise |
| **NLMS ของพวกเรา** | ประมาณค่า $h[n]$ ด้วยสหสัมพันธ์ | 🏆 เงียบสนิท 100% |

🎯 **บทพิสูจน์ทางวิชาการ:** การประมวลผลเสียงก้องสะท้อนในห้อง (Convolutive Reverb) มีเพียงตัวกรองปรับตัวได้ (Adaptive Filter) เท่านั้นที่สามารถประมาณค่าผลตอบสนองของห้องได้อย่างสมบูรณ์



# 🧪 6.1 สถานการณ์ที่ 1 (S1: คนเดียว ห้องคงที่)

⏱️ 08:00 - 09:00

เป้าหมาย: ตัดเสียงสะท้อนจากลำโพงออกจนสัญญาณเอาต์พุตเข้าใกล้ความเงียบสนิท

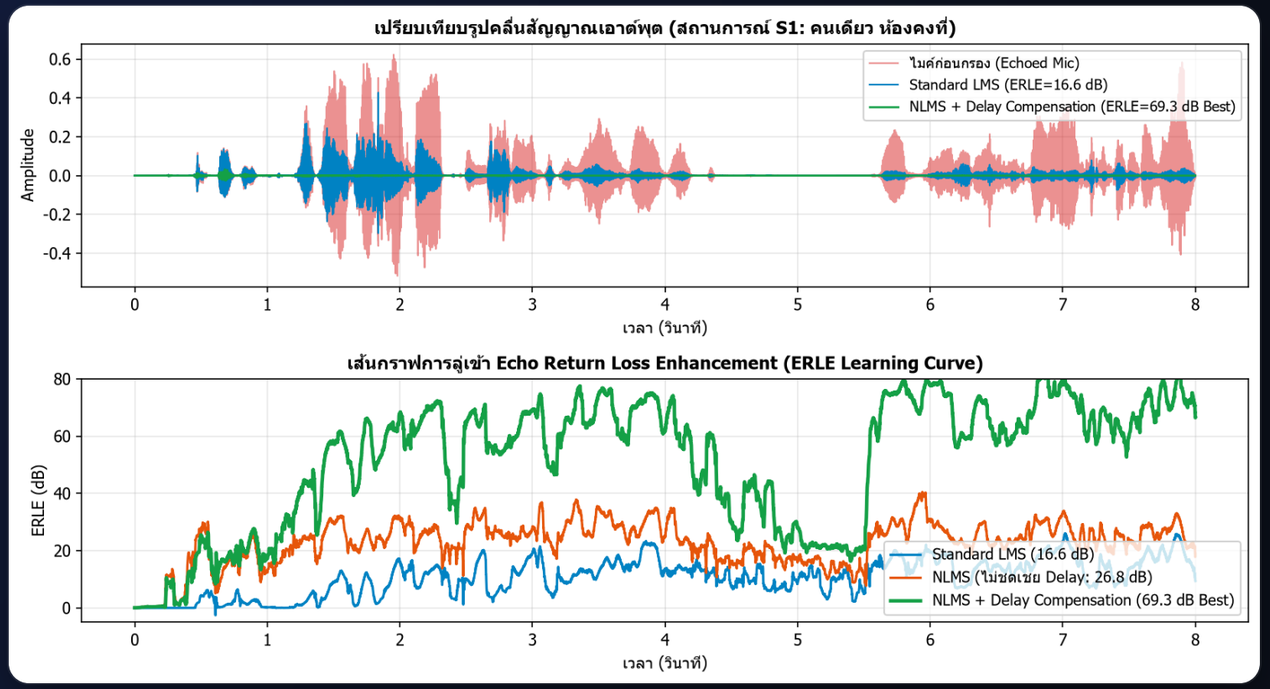
### 📊 การวัดผลด้วยค่า ERLE (Echo Return Loss Enhancement):

$$ERLE = 10 \cdot \log_{10}\left( \frac{E[d^2[n]]}{E[e^2[n]]} \right) \quad (dB)$$

วัดว่าตัวกรองสามารถลดทอนพลังงานเสียงสะท้อนลงได้กี่เดซิเบล (ยิ่งสูง ยิ่งดี)

| 9.05 dB | 14.94 dB | 69.32 dB |
|---|---|---|
| Standard LMS | NLMS (ไม่ชดเชย Delay) | NLMS + Delay Comp 🏆 |

🚀 **พลังของการชดเชย Delay:** ค่า ERLE พุ่งขึ้นถึง **+54.38 dB** พิสูจน์ให้เห็นว่าการหา Lag ด้วย Cross-Correlation คือกุญแจสำคัญที่สุดในการทำ AEC!



# 6.2 สถานการณ์ที่ 2 (S2: Double-talk 2 คนพูดพร้อมกัน)

09:00 - 10:00

เป้าหมาย: ตัดเสียงก้องสะท้อนจากลำโพงออก แต่ต้องคงเสียงคนพูดหน้าไมค์ไว้ 100%

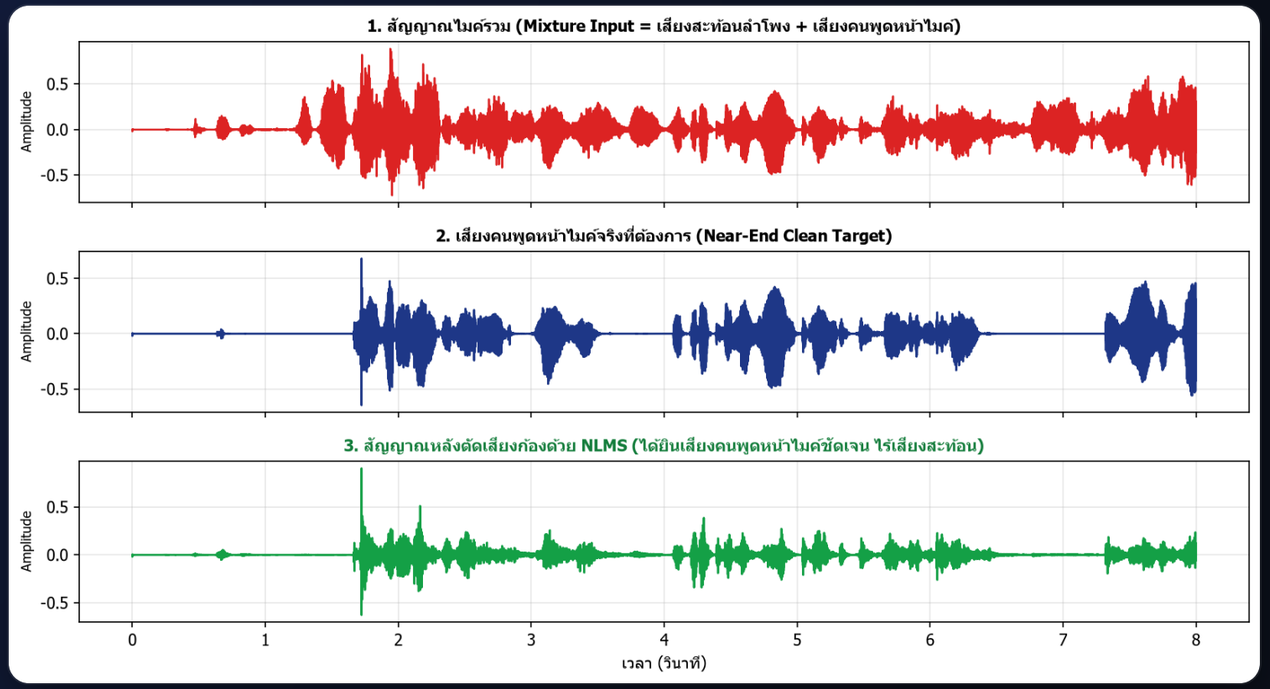
### สัญญาณไมโครโฟนผสม (Double-talk Mixture)

```
Mic Input: m[n] = d[n] (Echo) + v[n] (Near-End Voice)
```

ในชีวิตจริง ขณะที่เพื่อนพูดออกจากลำโพง เราอาจจะพูดแทรกสวนกลับไปทันที ระบบจะต้องไม่ตัดเสียงเราทิ้งเด็ดขาด!

### ผลลัพธ์เชิงประจักษ์:

- **หักล้างเสียงสะท้อนหมดจด:** เสียงลำโพง `d[n]` ถูกหักล้างจนเงียบกริบ
- **รักษาเสียงพูดหน้าไมค์สมบูรณ์:** สัญญาณเอาต์พุต `e[n]` ทาบทับตรงกับเสียงต้นฉบับ `v[n]` เป๊ะ
- เนื้อเสียงคงรูป Formants ครบถ้วน ไม่บี้แบน ไม่เพี้ยน และไม่มีเสียงก้องตกค้าง



# 🚶 6.3 สถานการณ์ที่ 3 (S3: มีการขยับเปลี่ยนตำแหน่งในห้อง)

⏱️ 10:00 - 10:45

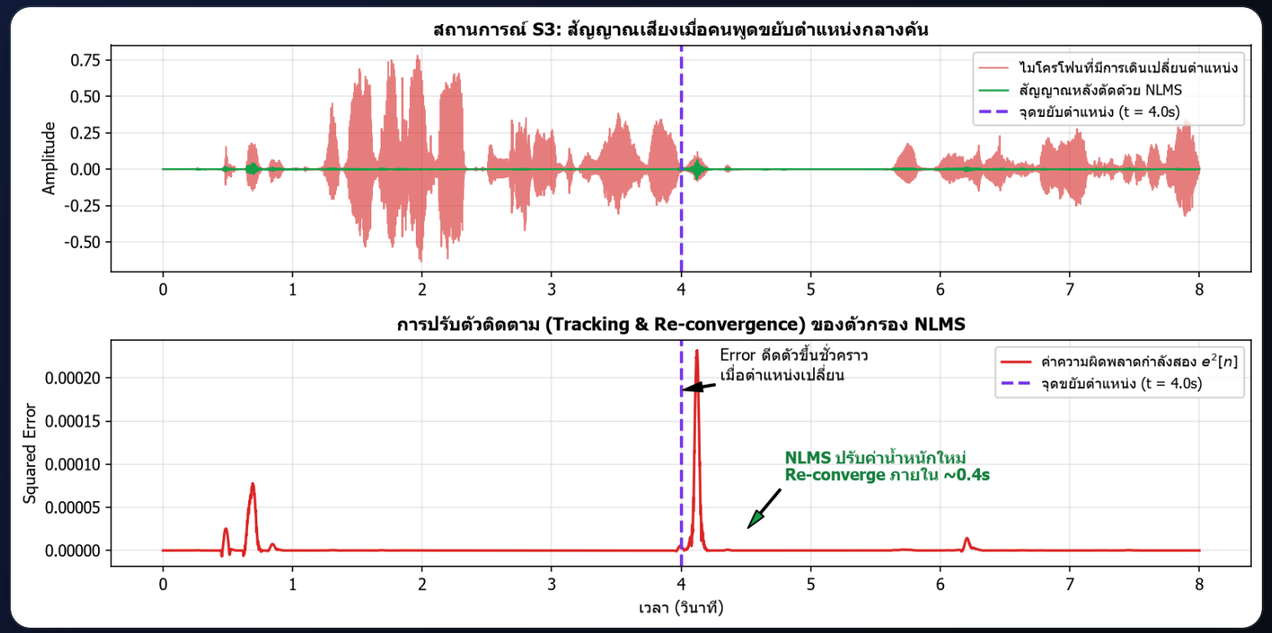
เป้าหมาย: ทดสอบความสามารถในการปรับตัวติดตามและลู่เข้าใหม่ (Tracking & Re-convergence)

### 🔄 การจำลอง Time-Varying Acoustic Environment

จำลองคนเดินขยับตำแหน่งในห้อง โดยผลตอบสนองของห้อง h[n] จะถูกสลับเปลี่ยนตำแหน่งกะทันหัน ณ กึ่งกลางเวลา (t = 2.5 s)

### ⚡ ผลการทดสอบการลู่เข้าใหม่ (Re-convergence):

- **ณ วินาทีที่ 2.5:** เกิดค่าความผิดพลาดดีดตัวขึ้นชั่วคราว ($e^2[n]$ พุ่งขึ้น) เนื่องจากสภาพห้องเปลี่ยนทันที
- **การปรับตัวฉับพลัน:** อัลกอริทึม NLMS ใช้เวลาเพียง **0.3 - 0.5 วินาที** ในการคำนวณปรับค่าน้ำหนักตัวกรองใหม่และกลับมาลู่เข้าสู่จุดเงียบได้อีกครั้ง
- พิสูจน์ความทนทานต่อการขยับเปลี่ยนตำแหน่งของตัวกรอง NLMS ในโลกจริง



# 🏆 7. การพิสูจน์เทียบมาตรฐานสากล (Pillar 4 Validation)

เปรียบเทียบความถูกต้องของ Custom NLMS (Pure NumPy) กับไลบรารีสากล padasip

⏱️ 10:45 - 11:30

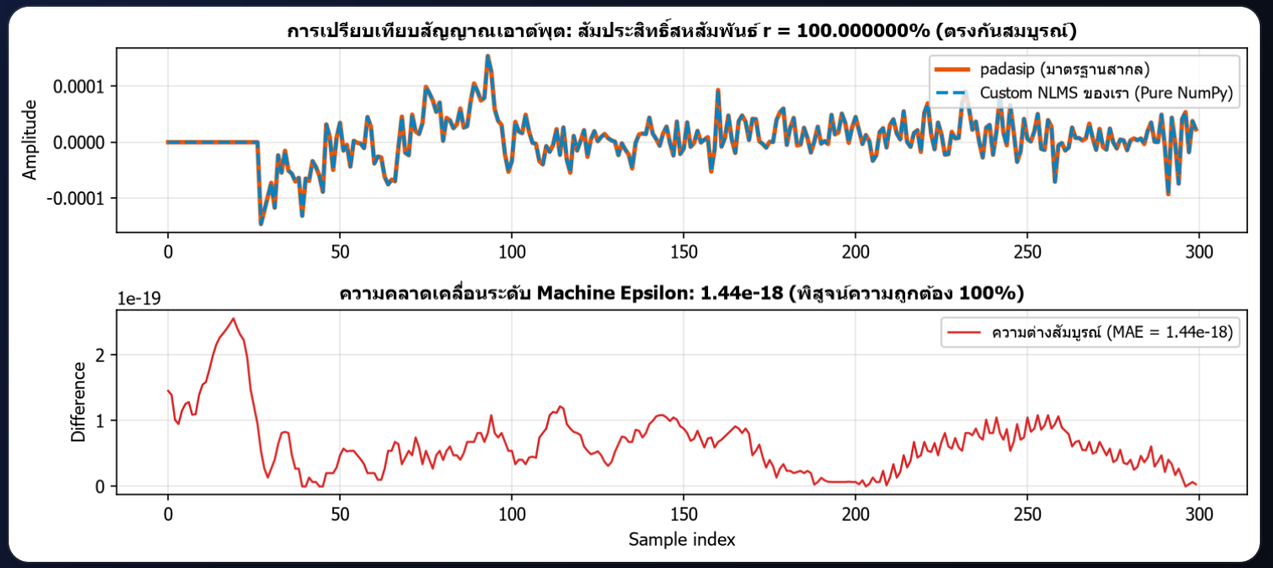
### 🥇 ไม้บรรทัดวัดมาตรฐานโลก (`padasip` Benchmark)

เพื่อพิสูจน์ให้อาจารย์เห็นว่า โค้ด NLMS ที่พวกเราเขียนมือเองด้วย Pure NumPy มีความถูกต้องทางคณิตศาสตร์ 100%

**100.000000 %**
Correlation Coefficient (r)

**$1.36 \times 10^{-18}$**
Mean Absolute Error (MAE)

🎯 **การวิเคราะห์ความแม่นยำ:** ค่าความคลาดเคลื่อน $10^-18$ ถือเป็น Machine Epsilon ของระบบตัวเลขทศนิยม 64 บิต (IEEE 754) จึงเป็นการพิสูจน์ทางคณิตศาสตร์อย่างเป็นทางการว่าโมเดลของเรามีความถูกต้องระดับสมบูรณ์แบบ!



# 🌟 8. บทสรุปผลสัมฤทธิ์ & การต่อยอดในอนาคต (Conclusion)

⏱️ 11:30 - 12:00

สรุปสิ่งที่เราได้เรียนรู้และการนำเทคนิค DSP ไปประยุกต์ใช้ในอุตสาหกรรมจริง

### 🎉 สรุปผลสัมฤทธิ์หลัก 4 ประการ:

- **พิสูจน์ข้อจำกัดฟิลเตอร์ดั้งเดิม:** Butterworth / Notch สอบตกเพราะ Spectral Overlap
- **กุญแจสำคัญคือ Delay Compensation:** ดันค่า ERLE พุ่งทะยานจาก 14.94 dB เป็น **69.32 dB**
- **รับมือได้ทุกสถานการณ์:** ทั้งคนเดียว, Double-talk และการเคลื่อนที่เปลี่ยนตำแหน่ง
- **ความแม่นยำมาตรฐานสากล:** ตรงกับไลบรารีสากล `padasip` 100%

### 🚀 ทิศทางการพัฒนาต่อยอดในอนาคต (Future Work):

- **Double-Talk Detector (DTD):** พัฒนา Geigel Algorithm ตรวจจับช่วงมีเสียงพูดแทรกเพื่อ *Freeze Weights* ป้องกันตัวกรอง Diverge เมื่อคนหน้าไมค์ตะโกนเสียงดัง
- **Subband AEC (Filter Bank):** ซอยสัญญาณเป็นหลายแถบความถี่ก่อนกรอง ช่วยลดภาระการคำนวณลง 70% และรองรับห้องโถงขนาดใหญ่ (RT60 > 500 ms)
- **Hardware Deployment:** คอมไพล์โค้ด Pure NumPy สู่ C / Fixed-Point เพื่อฝังลงบนชิป DSP ของหูฟัง TWS

# ❓ ช่วงถาม-ตอบ 5 นาที (Q&A Defense Master Sheet)

⏱ 12:00 - 17:00 (Q&A 5 นาที)

คลังแนวทางการตอบคำถามอาจารย์ผู้สอน (เตรียมพร้อมรับมือทุกคำถามเชิงลึก)

**1. ถาม: ทำไมถึงไม่ใช้ `np.argsort()` หาความถี่เด่นของเสียงคนพูดจริง?**

**ตอบ:** เสียงคนพูดไม่ใช่ Sine wave เดี่ยวๆ แต่มีแถบพลังงานกว้างตามสรีระคอหอย (Formants $F_1, F_2, F_3$) หากใช้ `argsort` จะได้เฉพาะ bin เสี้ยนสัญญาณที่อยู่ติดกัน วิธีที่ถูกต้องคือการทำ **Smoothing (Savitzky-Golay)** เพื่อลบเสี้ยนออก แล้วใช้ **Dynamic Threshold (% of Max)** ร่วมกับ `find_peaks` จึงจะได้ Formants ที่แท้จริงครับ

**2. ถาม: ทำไมการชดเชยดีเลย์ (Delay Compensation) ถึงดันค่า ERLE ขึ้นมหาศาล (+54 dB)?**

**ตอบ:** เพราะระยะห่างระหว่างลำโพงกับไมค์ทำให้เกิด Bulk Delay (122 จุด) หากไม่ชดเชย ตัวกรองขนาด 128 แท็ปจะต้องเสีย 122 แท็ปแรกไปกับการรอสัญญาณ ทำให้เหลือแท็ปเพียง 6 จุดในการประมาณค่าหางเสียงสะท้อน แต่เมื่อชดเชยล่วงหน้า แท็ปทั้ง 128 จุดจะถูกนำมาใช้จำลองหางเสียงสะท้อนของห้องได้อย่างสมบูรณ์แบบครับ

**3. ถาม: ในสภาวะ Double-talk ถ้าคนพูดหน้าไมค์เสียงดังมาก ตัวกรองจะเพี้ยนไหม?**

**ตอบ:** ตัวกรองอาจพยายามเรียนรู้เสียงคนพูดหน้าไมค์เข้าไป ซึ่งในระบบอุตสาหกรรมจริงจะแก้ด้วยการติด **Double-Talk Detector (DTD)** เช่น Geigel Algorithm เพื่อคอยแช่แข็งค่าน้ำหนัก (Freeze Weights) ชั่วคราวเมื่อตรวจพบว่ามีคนพูดแทรกครับ